ldapa-plan

LDAPA Intelligent Portal — Project Specification

Version: 1.0
Date: February 16, 2026
Team: Haya Al Kubaisi, Anel Adalbek, Noor Niknam, Hazem Elsayed
Advisor: Dr. Collin Diedrich

---

1. Project Overview

Two separate web applications backed by a single shared database:

App	Audience	Purpose
Public Portal	Parents, adults with LDs, educators, caregivers	Landing page + LLM-powered chat that provides plain-language guidance and provider recommendations
Admin Panel	LDAPA staff (~1–3 users)	Authenticated dashboard for managing the provider directory and monitoring chat quality

---

2. Feature Set

2.1 Public Portal

#	Feature	Description
F1	**Landing Page**	Hero section explaining what LDAPA is and what this tool does. Four quick-start cards that seed the chat with a pre-filled prompt. About section. Footer with LDAPA contact info and disclaimer.
F2	**Quick-Start Cards**	Pre-built entry points that open the chat with a starter message: (1) "My child is struggling in school" (2) "I think I may have a learning disability" (3) "I'm a teacher looking for resources" (4) "I need an affordable evaluation"
F3	**Chat Interface**	Full-page conversational UI. User types in plain language. LLM responds with guidance, educational info, and provider recommendations. The LLM naturally asks for the user's

#	Feature	Description
		city/town/ZIP early in the conversation to personalize provider results.
F4	**Provider Cards (inline)**	When the LLM recommends providers, structured cards render inline in the chat. Each card shows: name, service type, location, cost tier, phone/website. Cards have a collapsed and expanded state.
F5	**Disclaimer Footer**	Persistent footer on every chat screen: "This tool provides general information only. It does not diagnose conditions or provide legal or medical advice. Please consult a qualified professional for specific guidance."
F6	**Escalation to LDAPA**	When the LLM detects a query beyond its scope (crisis, legal, diagnostic), it surfaces LDAPA's contact info prominently with a message like "This is something best handled by a person — here's how to reach LDAPA directly."
F7	**Feedback Thumbs**	Thumbs up / thumbs down on each LLM response. Stored for quality monitoring in the admin dashboard.
F8	**Conversation Reset**	"Start over" button that clears the chat and returns the user to a fresh state.

2.2 Admin Panel

#	Feature	Description
A1	**Login**	Email/password authentication. Single admin role (no RBAC).
A2	**Dashboard**	Overview stats: total providers (verified/unverified/archived), chat sessions today/this week/this month, average feedback score, top 5 query themes.
A3	**Provider List**	Searchable, sortable, filterable table. Columns: name, service type, city, status, last verified. Filters: status (verified/unverified/archived), service type, city/region. Bulk actions: verify selected, archive selected.
A4	**Add Provider**	Form with all provider fields. Saves as "unverified" by default.
A5	**Edit Provider**	Pre-populated form. Can update any field and change verification status.
A6	**Provider Detail View**	Read-only view of full provider record with action buttons: Edit, Verify, Archive, Delete (soft). Includes internal staff notes.
A7	**Bulk Import**	CSV upload → preview table showing parsed records with validation warnings → confirm to import.

#	Feature	Description
A8	**Recent Chat Review**	List of recent anonymized chat sessions. Click to expand and read the full conversation. Shows feedback ratings per message. Purpose: quality review and identifying directory gaps.

---

3. Screen Inventory (Figma Designs)

3.1 Public Portal Screens

Screen	Key Elements	States/Variants
S1: Landing Page	Navbar (logo, "Start Chat" CTA), hero section with headline + subtext, 4 quick-start cards in a grid, "How it works" 3-step section, about LDAPA section, footer with disclaimer + contact	Desktop, Tablet, Mobile
S2: Chat Interface	Left: conversation area with message bubbles (user = right-aligned, assistant = left-aligned). Input bar at bottom with send button. Top bar with LDAPA logo + "Start Over" button. Persistent disclaimer.	Empty state (welcome message), Active conversation, Provider cards rendered inline
S3: Provider Card Component	Collapsed: name, service type, city, cost tier badge. Expanded: all fields (phone, email, website, description, ages served, specializations, insurance info).	Collapsed, Expanded
S4: Escalation State	Chat shows a visually distinct "contact LDAPA" card with phone, email, and hours. Differentiated from regular provider cards.	—

3.2 Admin Panel Screens

Screen	Key Elements	States/Variants
S5: Login	Centered card with email + password fields, login button, LDAPA branding.	Default, Error state
S6: Dashboard	Stat cards row (total providers, verified %, chats this week, avg feedback). Bar chart of chats over time. List of top query themes.	—

Screen	Key Elements	States/Variants
S7: Provider List	Search bar, filter dropdowns (status, service type, region), sortable table, bulk action toolbar, pagination. "Add Provider" button top-right.	Empty state, Populated, Filtered
S8: Add/Edit Provider Form	Two-column form layout. Left: name, organization, service types (multi-select), specializations (multi-select), cost tier, insurance fields. Right: address fields, contact info, description textarea, serves ages (multi-select). Bottom: save/cancel buttons.	Add mode (blank), Edit mode (pre-filled)
S9: Provider Detail View	Full record displayed read-only in a card layout. Action buttons: Edit, Verify, Archive, Delete. Internal notes section (not visible to public). Metadata: created date, last verified date, last updated.	Verified state, Unverified state
S10: Bulk Import	Step 1: File upload dropzone. Step 2: Preview table with validation (green rows = valid, yellow = warnings, red = errors). Step 3: Confirmation + import.	Upload, Preview, Success/Error
S11: Chat Review	List of recent sessions (date, message count, avg feedback). Click to expand full conversation transcript. Each message shows feedback rating if given.	List view, Expanded session

Total: 11 screens + mobile/tablet responsive variants for S1 and S2.

---

4. Technical Architecture

4.1 Stack

Layer	Technology	Rationale
Public Portal Frontend	Next.js (App Router) on Vercel	Fast static landing page + dynamic chat. Free tier handles expected traffic.
Admin Panel Frontend	Next.js (App Router) on Vercel (separate project)	Auth-gated. Same framework = shared team knowledge.
Backend API	Python FastAPI on Railway	Single API server shared by both frontends. Python ecosystem for LLM integration. Easy to maintain. Free/cheap tier on Railway.

Layer	Technology	Rationale
Database	PostgreSQL on Supabase	Free tier handles hundreds of records. Managed hosting. LDAPA owns the account. Built-in auth for admin users.
Auth	Supabase Auth	Email/password, single admin role. Integrated with the DB. Free tier.
LLM	Claude API (Sonnet) or Gemini Flash	Both are cost-effective at 5K–20K queries/year. Claude Sonnet preferred for empathetic, nuanced conversation.
Search	PostgreSQL full-text search (`tsvector`)	At hundreds of records, Postgres FTS + structured SQL filters is sufficient. Eliminates Elasticsearch dependency.

4.2 Architecture Diagram

```
┌──────────────────────┐          ┌──────────────────────┐
│    PUBLIC PORTAL     │          │     ADMIN PANEL      │
│   Next.js (Vercel)   │          │   Next.js (Vercel)   │
│                      │          │                      │
│  - Landing Page      │          │  - Login             │
│  - Chat Interface    │          │  - Dashboard         │
│                      │          │  - Provider CRUD     │
│  Calls backend API → │          │  - Bulk Import       │
│                      │          │  - Chat Review       │
│                      │          │                      │
│                      │          │  Calls backend API → │
└──────────┬───────────┘          └──────────┬───────────┘
           │                                 │
           │    ┌──────────────────────┐     │
           └──→ │   FastAPI Backend    │ ←───┘
                │   (Railway)          │
                │                      │
                │  /api/chat           │
                │  /api/feedback       │
                │  /api/admin/*        │
                └──────┬──────┬────────┘
                       │      │
                ┌──────┘      └──────┐
                ▼                    ▼
        ┌──────────────┐      ┌──────────────┐
        │ PostgreSQL   │      │  LLM API     │
        │ (Supabase)   │      │ (Claude or   │
```

```
        │              │      │  Gemini)    │
        └──────────────┘      └─────────────┘
```

4.3 Chat Flow (Two-LLM-Call Pattern)

Each user message triggers this sequence:

```
User sends message
       │
       ▼
┌──────────────────────────────────────────┐
│ CALL 1: Filter Extraction                │
│                                          │
│ Input: conversation history +            │
│        latest user message               │
│                                          │
│ System prompt: "Extract structured       │
│ search filters from this                 │
│ conversation. Return JSON:               │
│ {                                        │
│   service_types: [],                     │
│   specializations: [],                   │
│   cost_tier: [],                         │
│   location: { city, zip },               │
│   age_group: [],                         │
│   needs_providers: boolean,              │
│   escalate: boolean                      │
│ }"                                       │
│                                          │
│ Output: structured JSON filters          │
└──────────────────────────────────────────┘
                │
                │
                ▼
┌──────────────────────────────────────────┐
│ DATABASE QUERY                           │
│                                          │
│ If needs_providers == true:              │
│   Query Postgres with extracted          │
│   filters + full-text search             │
│   Return top 5 matching providers        │
│                                          │
│ If escalate == true:                     │
│   Skip DB, prepare escalation            │
└──────────────────────────────────────────┘
                │
```

```
                ▼
┌──────────────────────────────────────┐
│ CALL 2: Response Generation          │
│                                      │
│ Input: system prompt +               │
│        conversation history +        │
│        provider results (if any)     │
│                                      │
│ System prompt includes:              │
│  - LDAPA mission and tone guidelines │
│  - Plain language mandate            │
│  - No diagnosis / no legal advice    │
│  - Disclaimer requirements           │
│  - Escalation triggers               │
│  - How to present provider recs      │
│                                      │
│ Output: natural language response +  │
│         structured provider list     │
└──────────────────────────────────────┘
```

---

5. Database Schema

5.1 Tables

```
-- ============================================
-- PROVIDERS
-- ============================================
CREATE TABLE providers (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            TEXT NOT NULL,
    organization    TEXT,

    -- Service classification
    service_types   TEXT[] NOT NULL,          -- e.g., {'evaluator', 'tutor',
'advocate', 'therapist', 'clinic', 'support_group'}
    specializations TEXT[],                   -- e.g., {'dyslexia', 'adhd',
'dyscalculia', 'general_ld', 'adult_ld'}
    serves_ages     TEXT[],                   -- e.g., {'children',
'adolescents', 'adults'}

    -- Location
    address         TEXT,
    city            TEXT NOT NULL,
```

```
    state           TEXT NOT NULL DEFAULT 'PA',
    zip_code        TEXT,
    region          TEXT,                     -- e.g., 'Pittsburgh Metro',
'Rural Western PA'

    -- Cost information
    cost_tier       TEXT NOT NULL,            -- 'free', 'sliding_scale',
'low_cost', 'standard'
    insurance_accepted  BOOLEAN DEFAULT FALSE,
    accepts_medicaid    BOOLEAN DEFAULT FALSE,
    cost_notes      TEXT,                     -- e.g., "Free for families
under $50k income"

    -- Contact
    phone           TEXT,
    email           TEXT,
    website         TEXT,

    -- Content (used for RAG context)
    description     TEXT,                     -- Plain language description of
services

    -- Governance
    verification_status TEXT NOT NULL DEFAULT 'unverified',  -- 'verified',
'unverified', 'archived'
    last_verified_at    TIMESTAMPTZ,
    staff_notes     TEXT,                     -- Internal notes, not shown to
users

    -- Full-text search
    search_vector   TSVECTOR GENERATED ALWAYS AS (
        to_tsvector('english',
            coalesce(name, '') || ' ' ||
            coalesce(organization, '') || ' ' ||
            coalesce(description, '') || ' ' ||
            coalesce(city, '') || ' ' ||
            coalesce(array_to_string(service_types, ' '), '') || ' ' ||
            coalesce(array_to_string(specializations, ' '), '')
        )
    ) STORED,

    -- Metadata
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    is_deleted       BOOLEAN NOT NULL DEFAULT FALSE          -- Soft delete
);
```

```sql
CREATE INDEX idx_providers_search ON providers USING GIN (search_vector);
CREATE INDEX idx_providers_city ON providers (city);
CREATE INDEX idx_providers_zip ON providers (zip_code);
CREATE INDEX idx_providers_status ON providers (verification_status);
CREATE INDEX idx_providers_service_types ON providers USING GIN
(service_types);


-- ============================================
-- CHAT SESSIONS (anonymous)
-- ============================================
CREATE TABLE chat_sessions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_location   JSONB,                  -- { city, zip } if provided,
else null
    started_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_message_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    message_count   INT NOT NULL DEFAULT 0,
    escalated       BOOLEAN NOT NULL DEFAULT FALSE
);


-- ============================================
-- CHAT MESSAGES
-- ============================================
CREATE TABLE chat_messages (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES chat_sessions(id) ON DELETE
CASCADE,
    role            TEXT NOT NULL,           -- 'user' or 'assistant'
    content         TEXT NOT NULL,
    providers_shown UUID[],                 -- Array of provider IDs surfaced
in this message
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_messages_session ON chat_messages (session_id, created_at);


-- ============================================
-- CHAT FEEDBACK
-- ============================================
CREATE TABLE chat_feedback (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    message_id      UUID NOT NULL REFERENCES chat_messages(id) ON DELETE
```

```
CASCADE,
    session_id      UUID NOT NULL REFERENCES chat_sessions(id) ON DELETE
CASCADE,
    rating          TEXT NOT NULL,            -- 'up' or 'down'
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_feedback_session ON chat_feedback (session_id);
```

5.2 Provider Field Reference

Field	Type	Required	Notes
`name`	text	✅	Provider or individual name
`organization`	text		Parent org if applicable
`service_types`	text[]	✅	Allowed: `evaluator`, `tutor`, `advocate`, `therapist`, `school_psychologist`, `clinic`, `support_group`, `nonprofit_org`
`specializations`	text[]		Allowed: `dyslexia`, `adhd`, `dyscalculia`, `dysgraphia`, `general_ld`, `adult_ld`, `iep_504`, `workplace_accommodations`
`serves_ages`	text[]		Allowed: `children`, `adolescents`, `adults`
`city`	text	✅	
`state`	text	✅	Default: PA
`zip_code`	text		5-digit ZIP
`region`	text		For grouping (e.g., "Greater Pittsburgh")
`cost_tier`	text	✅	Allowed: `free`, `sliding_scale`, `low_cost`, `standard`
`insurance_accepted`	bool		Default: false
`accepts_medicaid`	bool		Default: false
`cost_notes`	text		Free-text cost details
`phone`	text		
`email`	text		
`website`	text		
`description`	text		Used for RAG — should be detailed
`verification_status`	text	✅	`verified`, `unverified`, `archived`

Field	Type	Required	Notes
`last_verified_at`	timestamp		Set when staff marks verified
`staff_notes`	text		Internal only

---

6. API Endpoints (FastAPI)

Base URL: `https://api.ldapa-portal.org` (Railway deployment)

6.1 Public Endpoints

Method	Endpoint	Request Body	Response	Description
`POST`	`/api/chat`	`{ session_id?: string, message: string, history: Message[] }`	`{ session_id: string, response: string, providers: Provider[], escalate: boolean }`	Main chat endpoint. Creates session if none. Runs two-LLM-call flow. Stores messages.
`POST`	`/api/feedback`	`{ message_id: string, session_id: string, rating: "up" | "down" }`	`{ success: boolean }`	Store feedback on an assistant message.
`GET`	`/api/health`	—	`{ status: "ok" }`	Health check.

6.2 Admin Endpoints (all require `Authorization: Bearer <supabase_token>`)

Providers

Method	Endpoint	Request / Params
`GET`	`/api/admin/providers`	Query: `?search=&status=&service_type=&city=&`
`GET`	`/api/admin/providers/{id}`	—

Method	Endpoint	Request / Params
`POST`	`/api/admin/providers`	`ProviderCreate` body
`PUT`	`/api/admin/providers/{id}`	`ProviderUpdate` body
`DELETE`	`/api/admin/providers/{id}`	—
`PATCH`	`/api/admin/providers/{id}/verify`	—
`PATCH`	`/api/admin/providers/{id}/archive`	—
`POST`	`/api/admin/providers/bulk-verify`	`{ ids: string[] }`
`POST`	`/api/admin/providers/bulk-archive`	`{ ids: string[] }`

Bulk Import

Method	Endpoint	Request	Response
`POST`	`/api/admin/providers/import/preview`	CSV file upload (multipart)	`{ valid: ParsedProvider warnings: Warning[], errors: Error[`
`POST`	`/api/admin/providers/import/confirm`	`{ providers: ParsedProvider[] }`	`{ imported: in skipped: int }`

Dashboard

Method	Endpoint	Params	Response
`GET`	`/api/admin/dashboard/stats`	Query: `?period=week|month|all`	`{ total_pro int, veri int, unve int, arch int, chat_sess int, avg_feedb float, top_theme string[]`
`GET`	`/api/admin/dashboard/chat-volume`	Query: `?period=week|month`	`{ data: [ string, c int }] }`
`GET`	`/api/admin/dashboard/recent-sessions`	Query: `?page=1&per_page=20`	`{ session SessionSu total: in`
`GET`	`/api/admin/dashboard/sessions/{id}`	—	`{ session Session, messages: Message[] feedback: Feedback[`

6.3 Pydantic Models (FastAPI)

```python
# === Provider Models ===

class ProviderCreate(BaseModel):
    name: str
    organization: str | None = None
    service_types: list[str]
    specializations: list[str] = []
    serves_ages: list[str] = []
    address: str | None = None
    city: str
    state: str = "PA"
    zip_code: str | None = None
    region: str | None = None
```

```python
    cost_tier: str  # free | sliding_scale | low_cost | standard
    insurance_accepted: bool = False
    accepts_medicaid: bool = False
    cost_notes: str | None = None
    phone: str | None = None
    email: str | None = None
    website: str | None = None
    description: str | None = None
    staff_notes: str | None = None

class ProviderUpdate(BaseModel):
    # All fields optional for partial update
    name: str | None = None
    organization: str | None = None
    service_types: list[str] | None = None
    specializations: list[str] | None = None
    serves_ages: list[str] | None = None
    address: str | None = None
    city: str | None = None
    state: str | None = None
    zip_code: str | None = None
    region: str | None = None
    cost_tier: str | None = None
    insurance_accepted: bool | None = None
    accepts_medicaid: bool | None = None
    cost_notes: str | None = None
    phone: str | None = None
    email: str | None = None
    website: str | None = None
    description: str | None = None
    verification_status: str | None = None
    staff_notes: str | None = None

class ProviderResponse(BaseModel):
    id: str
    name: str
    organization: str | None
    service_types: list[str]
    specializations: list[str]
    serves_ages: list[str]
    address: str | None
    city: str
    state: str
    zip_code: str | None
    region: str | None
    cost_tier: str
```

```python
    insurance_accepted: bool
    accepts_medicaid: bool
    cost_notes: str | None
    phone: str | None
    email: str | None
    website: str | None
    description: str | None
    verification_status: str
    last_verified_at: str | None
    staff_notes: str | None  # Only included in admin responses
    created_at: str
    updated_at: str


# === Chat Models ===

class ChatMessage(BaseModel):
    role: str  # 'user' | 'assistant'
    content: str

class ChatRequest(BaseModel):
    session_id: str | None = None
    message: str
    history: list[ChatMessage] = []

class ProviderCard(BaseModel):
    id: str
    name: str
    organization: str | None
    service_types: list[str]
    city: str
    zip_code: str | None
    cost_tier: str
    phone: str | None
    email: str | None
    website: str | None
    description: str | None

class ChatResponse(BaseModel):
    session_id: str
    response: str
    providers: list[ProviderCard]
    escalate: bool

class FeedbackRequest(BaseModel):
    message_id: str
```

```python
    session_id: str
    rating: str  # 'up' | 'down'


# === Dashboard Models ===

class DashboardStats(BaseModel):
    total_providers: int
    verified: int
    unverified: int
    archived: int
    chat_sessions: int
    avg_feedback: float
    top_themes: list[str]

class SessionSummary(BaseModel):
    id: str
    started_at: str
    message_count: int
    user_location: dict | None
    escalated: bool
    avg_rating: float | None
```

---

7. LLM System Prompts

7.1 Filter Extraction Prompt (Call 1)

```
You are a query analyzer for LDAPA, a learning disabilities nonprofit in
Pennsylvania.

Given a conversation between a user and an assistant, extract structured
search filters
to query a provider directory. Return ONLY valid JSON with no other text.

Schema:
{
  "service_types": [],        // From: evaluator, tutor, advocate, therapist,
school_psychologist, clinic, support_group, nonprofit_org
  "specializations": [],      // From: dyslexia, adhd, dyscalculia,
dysgraphia, general_ld, adult_ld, iep_504, workplace_accommodations
  "cost_tier": [],            // From: free, sliding_scale, low_cost, standard
  "location": {
    "city": null,             // User's city if mentioned
```

```
    "zip": null               // User's ZIP if mentioned
  },
  "age_group": [],            // From: children, adolescents, adults
  "needs_providers": false,   // true if the user's query would benefit from
provider recommendations
  "escalate": false,          // true if the user describes a crisis, asks for
diagnosis, or needs legal advice
  "search_text": ""           // Free-text keywords for full-text search
fallback
}

Rules:
- Only include filters explicitly supported by the conversation
- Set needs_providers to true when the user asks for help finding
someone/something
- Set escalate to true for: self-harm, abuse, crisis, explicit requests for
diagnosis or legal determination
- If the user hasn't mentioned location yet, leave location fields null
- Be conservative — only extract what's clearly stated or strongly implied
```

7.2 Response Generation Prompt (Call 2)

```
You are a friendly, knowledgeable guide for LDAPA (Learning Disabilities
Association
of Pennsylvania). You help parents, adults, educators, and caregivers
understand
learning disabilities and find appropriate support services.

YOUR ROLE:
- Explain learning disabilities, evaluation processes, IEPs, 504 plans,
accommodations,
  and support pathways in plain, jargon-free language
- Help users articulate their needs even when they don't know the right
terminology
- Recommend verified service providers from the LDAPA directory when relevant
- Be warm, empathetic, patient, and encouraging

YOUR BOUNDARIES — NEVER:
- Diagnose any condition or suggest a specific diagnosis
- Provide legal advice or legal determinations
- Provide medical advice
- Claim to be a doctor, lawyer, therapist, or any licensed professional
- Guarantee outcomes

WHEN PRESENTING PROVIDERS:
```

```
- Present them as options to explore, not prescriptions
- Note that the user should verify details directly with providers
- If cost or location matters, highlight relevant details
- If no providers match, say so honestly and suggest contacting LDAPA directly

ESCALATION — When the user describes:
- A crisis situation, self-harm, or abuse
- Something requiring a licensed professional's judgment
- A legal dispute or formal complaint
→ Respond with empathy, then clearly direct them to contact LDAPA directly
  or appropriate emergency services. Do not attempt to handle it.

LOCATION:
- Early in the conversation, naturally ask where the user is located (city or
ZIP)
  so you can personalize provider recommendations
- Don't demand it — ask conversationally

TONE:
- Use short sentences and simple words
- Avoid clinical jargon — if you must use a term, explain it immediately
- Validate the user's concerns before giving information
- Be encouraging: seeking help is a positive step

DISCLAIMER:
Always end your first response with: "Just so you know — I provide general
information to help you get started, not professional diagnosis or legal
advice.
For specific guidance, connecting with a qualified professional is always a
good idea."

PROVIDER CONTEXT:
{provider_context}
```

---

8. Project File Structure

```
ldapa/
├── backend/                          # FastAPI backend (Railway)
|   ├── app/
|   |   ├── main.py                   # FastAPI app, CORS, lifespan
|   |   ├── config.py                 # Environment variables, settings
|   |   ├── database.py               # Supabase/Postgres connection
|   |   ├── auth.py                   # Supabase JWT validation middleware
```

```
│   │   ├── routers/
│   │   │   ├── chat.py              # POST /api/chat, POST /api/feedback
│   │   │   ├── providers.py         # All /api/admin/providers/* routes
│   │   │   └── dashboard.py         # All /api/admin/dashboard/* routes
│   │   ├── services/
│   │   │   ├── llm.py               # LLM client (filter extraction +
response generation)
│   │   │   ├── provider_search.py   # Postgres query builder with FTS +
filters
│   │   │   └── csv_importer.py      # CSV parsing and validation
│   │   ├── models/
│   │   │   ├── provider.py          # Pydantic models for providers
│   │   │   ├── chat.py              # Pydantic models for chat
│   │   │   └── dashboard.py         # Pydantic models for dashboard
│   │   └── prompts/
│   │       ├── filter_extraction.py  # System prompt for Call 1
│   │       └── response_generation.py # System prompt for Call 2
│   ├── requirements.txt
│   ├── Dockerfile
│   └── railway.toml
│
├── public-portal/                   # Next.js public-facing app (Vercel)
│   ├── src/
│   │   ├── app/
│   │   │   ├── page.tsx             # Landing page
│   │   │   ├── chat/
│   │   │   │   └── page.tsx         # Chat interface
│   │   │   └── layout.tsx           # Root layout with disclaimer
│   │   ├── components/
│   │   │   ├── HeroSection.tsx
│   │   │   ├── QuickStartCards.tsx
│   │   │   ├── ChatWindow.tsx
│   │   │   ├── MessageBubble.tsx
│   │   │   ├── ProviderCard.tsx
│   │   │   ├── EscalationCard.tsx
│   │   │   ├── FeedbackThumbs.tsx
│   │   │   └── Disclaimer.tsx
│   │   └── lib/
│   │       └── api.ts               # API client functions
│   ├── next.config.js
│   └── vercel.json
│
├── admin-panel/                     # Next.js admin app (Vercel)
│   ├── src/
│   │   ├── app/
│   │   │   ├── login/
```

```
│   │   │   │   └── page.tsx          # Login screen
│   │   │   ├── dashboard/
│   │   │   │   └── page.tsx          # Dashboard
│   │   │   ├── providers/
│   │   │   │   ├── page.tsx          # Provider list
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx      # Add provider form
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── page.tsx      # Provider detail
│   │   │   │   │   └── edit/
│   │   │   │   │       └── page.tsx  # Edit provider form
│   │   │   │   └── import/
│   │   │   │       └── page.tsx      # Bulk import
│   │   │   ├── chats/
│   │   │   │   ├── page.tsx          # Recent sessions list
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx      # Session transcript
│   │   │   └── layout.tsx            # Admin layout with sidebar nav
│   │   ├── components/
│   │   │   ├── Sidebar.tsx
│   │   │   ├── ProviderTable.tsx
│   │   │   ├── ProviderForm.tsx
│   │   │   ├── StatsCards.tsx
│   │   │   ├── ChatVolumeChart.tsx
│   │   │   ├── ImportWizard.tsx
│   │   │   └── SessionViewer.tsx
│   │   └── lib/
│   │       ├── api.ts                # Admin API client
│   │       └── auth.ts               # Supabase auth helpers
│   ├── next.config.js
│   └── vercel.json
│
├── database/
│   ├── schema.sql                    # Full schema (copy from Section 5)
│   ├── seed.sql                      # Sample provider data for development
│   └── migrations/                   # Future schema changes
│
└── docs/
    ├── HANDOFF.md                    # Ownership transfer guide
    ├── ADMIN_GUIDE.md                # How to use the admin panel
    ├── DEPLOYMENT.md                 # How to deploy / redeploy
    ├── RUNBOOK.md                    # Monthly maintenance tasks
    └── ARCHITECTURE.md               # This document
```

9. Figma Design Plan

9.1 Design System Setup (Do First)

Before any screens, create a shared design system page with:

- **Colors:** Primary (LDAPA brand blue), secondary, success green, warning amber, error red, neutral grays, background white/off-white
- **Typography:** Heading 1–3, body, caption, button text. Use a clean sans-serif (Inter or similar). Ensure readability for users with learning disabilities (min 16px body).
- **Spacing:** 4px base grid, 8/16/24/32/48 spacing tokens
- **Components library:**
  - Button (primary, secondary, ghost, destructive) in default/hover/disabled states
  - Input field (text, textarea, select, multi-select, file upload) with label, placeholder, error, disabled states
  - Card (provider card collapsed/expanded, stat card, info card)
  - Table row (default, selected, hover)
  - Badge (verified green, unverified yellow, archived gray, cost tier badges)
  - Chat bubble (user, assistant)
  - Navigation (public navbar, admin sidebar)
  - Modal (confirmation dialogs)
  - Feedback thumbs component

9.2 Screen Design Order

Design in this order to build up components progressively:

Order	Screen	Why This Order
1	Design system + component library	Foundation for everything
2	S1: Public Landing Page (desktop)	Establishes brand, layout grid, navbar
3	S2: Chat Interface (desktop)	Core product screen — builds chat bubbles, input bar
4	S3: Provider Card (collapsed + expanded)	Reusable component for chat and admin
5	S4: Escalation state	Variant of chat
6	S1 + S2: Mobile responsive	Adapt the two public screens
7	S5: Admin Login	Simple screen, establishes admin visual language

Order	Screen	Why This Order
8	S6: Admin Dashboard	Stat cards, chart, establishes admin layout + sidebar
9	S7: Provider List	Table component, filters, bulk actions
10	S8: Add/Edit Provider Form	Form layout, multi-select fields
11	S9: Provider Detail View	Read-only card layout + action buttons
12	S10: Bulk Import	Multi-step wizard (upload → preview → confirm)
13	S11: Chat Review	Session list + transcript viewer

9.3 Figma Agent Prompt Guide

For each screen, provide the Figma agent with:

1. **Screen name and purpose** (from Section 3)
2. **Exact elements to include** (from the Key Elements column)
3. **Component references** (e.g., "use the provider card component from the library")
4. **Layout specification** (e.g., "two-column form, 60/40 split")
5. **States to design** (e.g., "empty state, populated, error")
6. **Accessibility notes** (e.g., "minimum 16px body text, high contrast, visible focus states")
7. **Content examples** (real provider data, real chat messages from use cases)

9.4 Sample Content for Designs

Sample chat conversation (for S2):

> **User:** Hi, my son is in 3rd grade and really struggling with reading. His teacher says he might need extra help but I don't know where to start. We're in Pittsburgh.
>
> **Assistant:** I'm glad you reached out — you're already taking a great first step by looking into this. When a child struggles with reading in early grades, there are a few things that can really help. One good starting point is asking the school for a reading evaluation — this is free and can help identify exactly where your son needs support. You can request this in writing to the school principal. Based on your location in Pittsburgh, here are some providers that might be a good fit:
>
> [Provider Card: Pittsburgh Reading Institute — Tutor — Free/Sliding Scale] [Provider Card: Dr. Sarah Chen — Evaluator — Accepts Medicaid]
>
> Would you like to know more about the evaluation process, or about any of these providers?

Sample provider records (for S3, S7, S9):

Field	Example 1	Example 2
name	Dr. Sarah Chen	Pittsburgh Reading Institute
organization	Child Development Associates	—
service_types	evaluator	tutor, nonprofit_org
specializations	dyslexia, adhd, general_ld	dyslexia, general_ld
serves_ages	children, adolescents	children
city	Pittsburgh	Pittsburgh
zip_code	15213	15232
cost_tier	low_cost	free
insurance_accepted	true	false
accepts_medicaid	true	false
phone	(412) 555-0142	(412) 555-0298
website	www.childdevassociates.example.com	www.pghreading.example.org
verification_status	verified	verified
description	Licensed psychologist specializing in learning disability evaluations for children and teens. Offers comprehensive assessments including cognitive, academic, and behavioral testing. Sliding scale available.	Nonprofit reading tutoring program for elementary students with learning disabilities. Uses evidence-based Orton-Gillingham approach. Free for families meeting income guidelines.

Sample dashboard stats (for S6):

- Total providers: 147 (89 verified, 42 unverified, 16 archived)
- Chat sessions this week: 34
- Average feedback: 4.2 / 5
- Top themes: "reading evaluation", "affordable tutoring", "IEP process", "adult ADHD assessment", "workplace accommodations"

---

10. Deployment & Handoff Checklist

10.1 Development Phase

- [ ] Set up Supabase project (team-owned during dev)
- [ ] Run schema.sql to create tables
- [ ] Seed sample provider data
- [ ] Deploy FastAPI backend to Railway (dev environment)
- [ ] Deploy public portal to Vercel (dev)
- [ ] Deploy admin panel to Vercel (dev)
- [ ] Configure CORS between frontends and backend
- [ ] Set up environment variables (LLM API key, Supabase URL/key)

10.2 Handoff to LDAPA

- [ ] Create LDAPA-owned Supabase account and migrate database
- [ ] Create LDAPA-owned Railway account and transfer backend
- [ ] Create LDAPA-owned Vercel account and transfer both frontends
- [ ] Create LDAPA-owned LLM API account with spending caps
- [ ] Set up custom domain (if applicable)
- [ ] Create admin user accounts for LDAPA staff
- [ ] Walk through admin panel with LDAPA staff
- [ ] Deliver documentation package (HANDOFF.md, ADMIN_GUIDE.md, DEPLOYMENT.md, RUNBOOK.md)
- [ ] Confirm LDAPA has password recovery set up for all accounts
- [ ] Confirm billing/credit card attached to all paid services with spending limits

10.3 Monthly Maintenance (LDAPA Responsibility Post-Handoff)

- [ ] Review and verify provider listings (quarterly at minimum)
- [ ] Check chat feedback for quality issues
- [ ] Review LLM API spending dashboard
- [ ] Update provider records as needed via admin panel
- [ ] Flag any system issues to contractor/student team if applicable

---

11. Success Metrics

Metric	Baseline	Target	How to Measure
Staff time on repetitive inquiries	High (estimated 10+ hrs/week)	25-30% reduction	Pre/post inquiry volume comparison

Metric	Baseline	Target	How to Measure
User self-navigation success	0% (no self-service tool)	60%+ of test users find a relevant resource without staff help	Task-based usability testing
Affordable option discovery	Difficult to identify	Users find 1+ affordable option within 2 min in test scenarios	Scenario-based testing
Directory accuracy	Fragmented, no verification	90%+ listings verified within defined cycles	Admin panel audit
Chat feedback score	N/A	80%+ positive (thumbs up)	Feedback data from dashboard

---

End of specification.